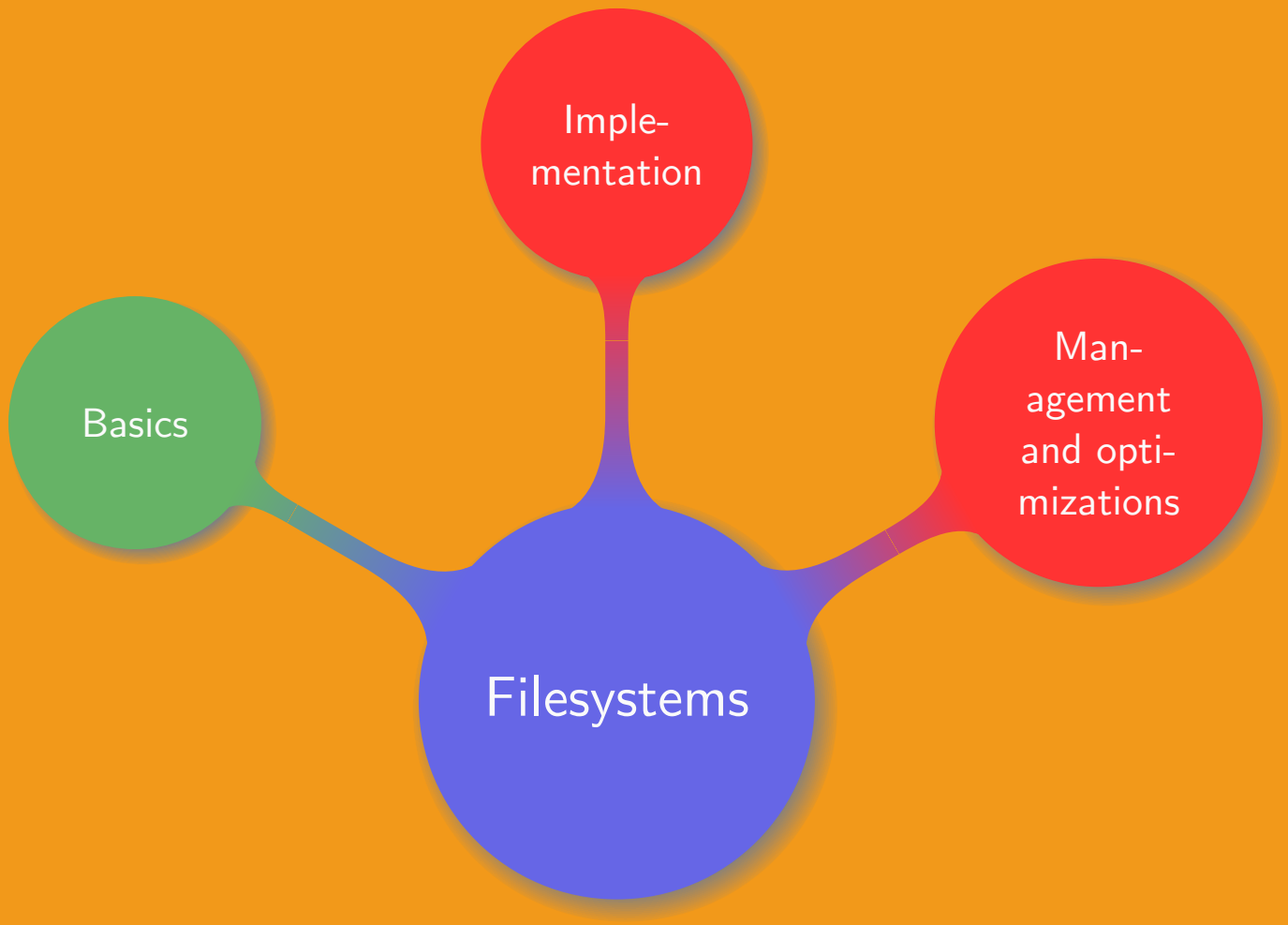
Introduction to Operating Systems

8. Filesystems

Manuel – Fall 2019

Chapter organisation



Goals

Limitations of virtual memory:

- Small
- Volatile
- Process dependent

Goals

Limitations of virtual memory:

- Small
- Volatile
- Process dependent

Goals that need to be achieved:

- Store large amount of data
- Long term storage
- Information shared among multiple processes

User's view

```
chattr +i file   // prevent from deleting it
```

High level view of a file-system:

- Small part of the disk memory can be directly accessed using high level abstraction called a *file*
- File name can be case sensitive or insensitive
- File name is a string with (an optional) suffix 后缀
- Each file has some attributes containing special information

File operations

Common system calls related to files (Unix):

- Create
- Delete
- Rename
- Open
- Close
- Read
- Write
- Append
- Seek
- Set attributes
- Get attributes

File organisation

Structure content:

- Files are grouped inside a *directory*
- Directories are organised in a *tree*
- Each file has an *absolute path* from the root of the tree
- Each file has an *relative path* from the current location in the tree

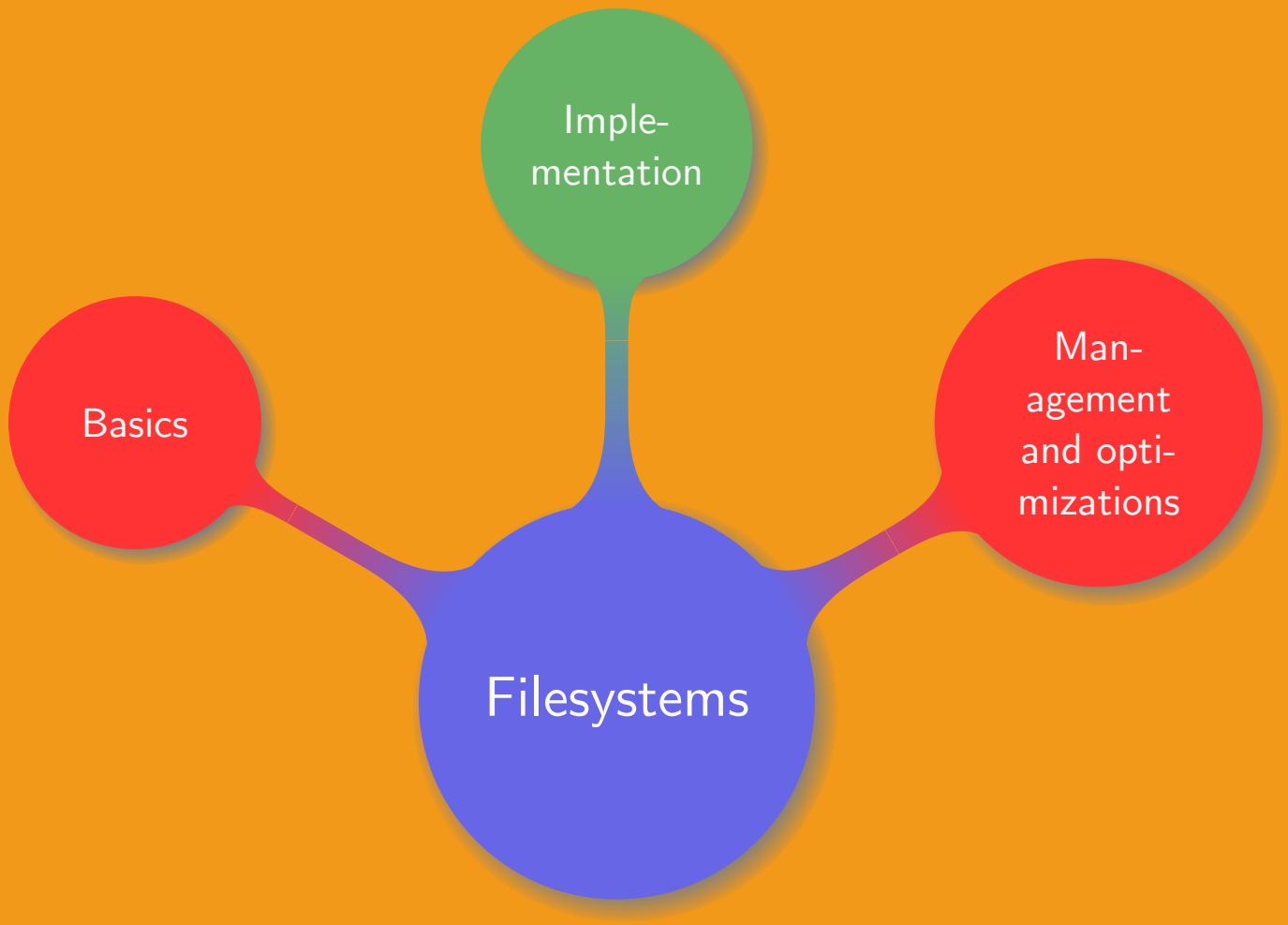
Directory operations

Common system calls related to directories (Unix):

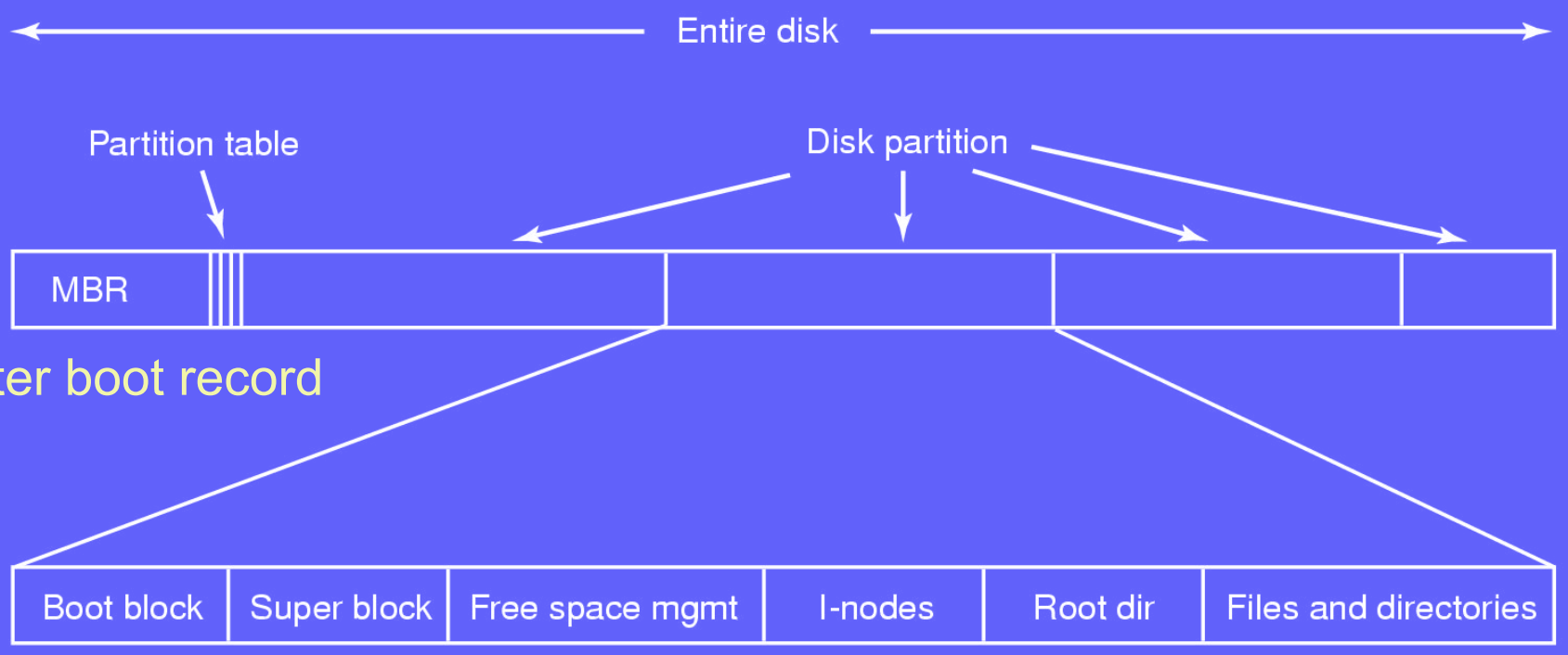
- Create
- Delete
- Opendir
- Closedir
- Readdir
- Rename
- Link
- Unlink

soft link/ hard link
"ln"?

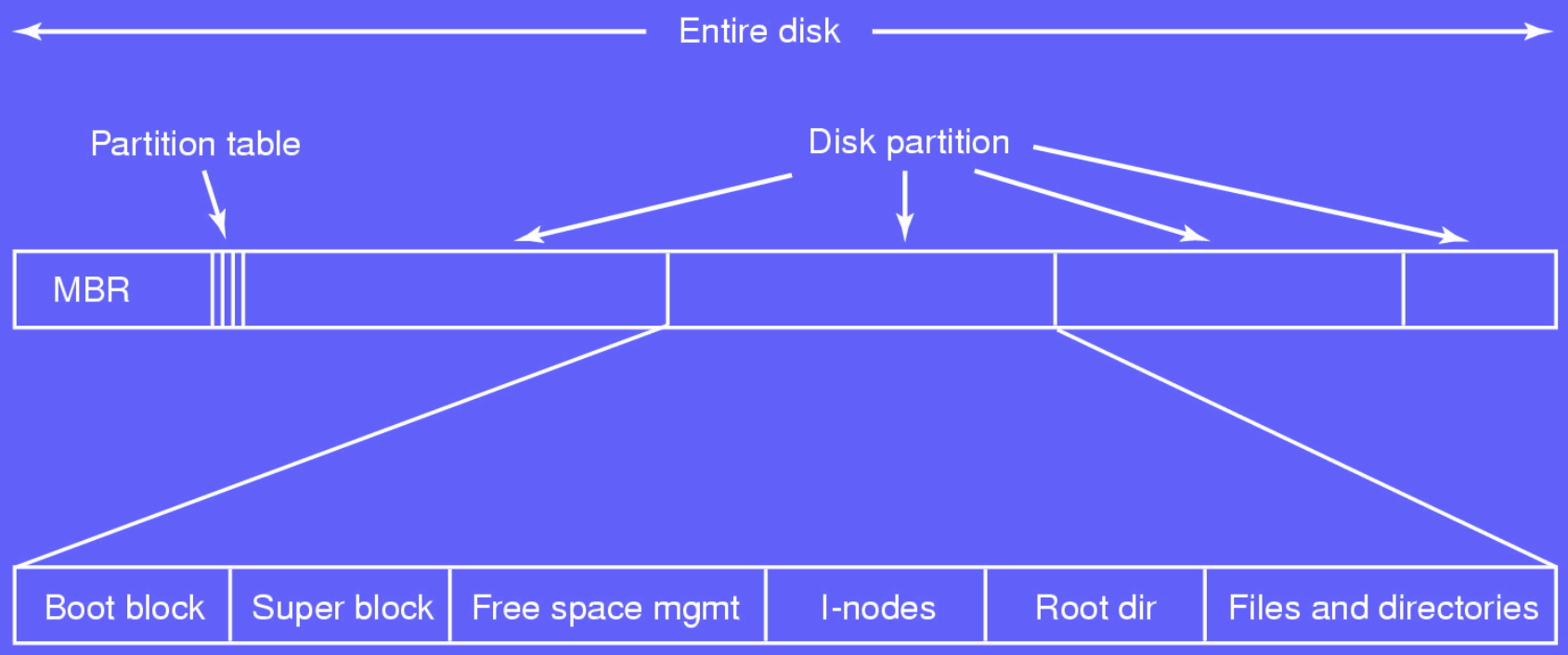
Chapter organisation



Basic disk layout



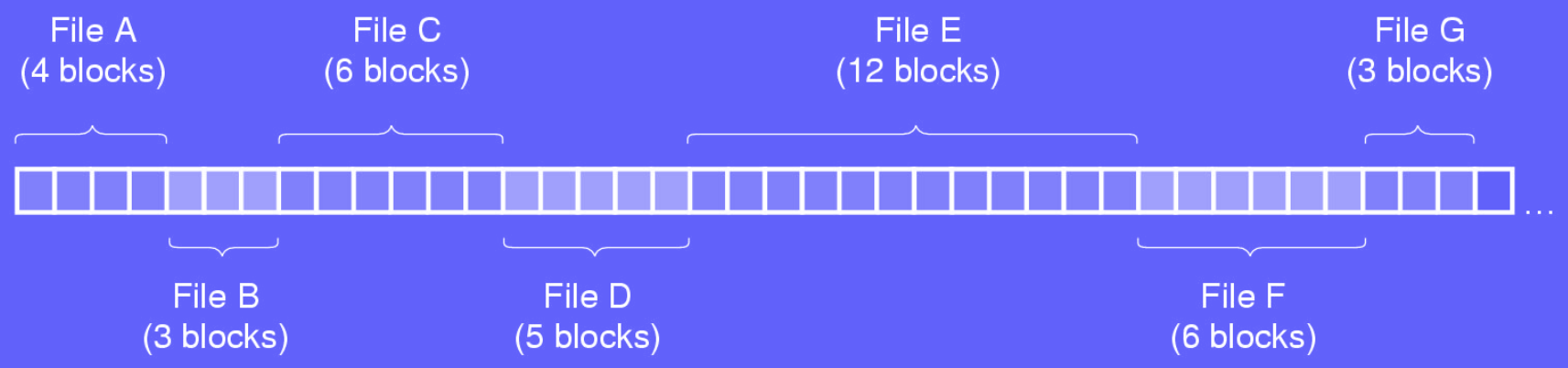
Basic disk layout



How to efficiently match disk blocks and files?

each file has some blocks. How to store and locate the blocks?

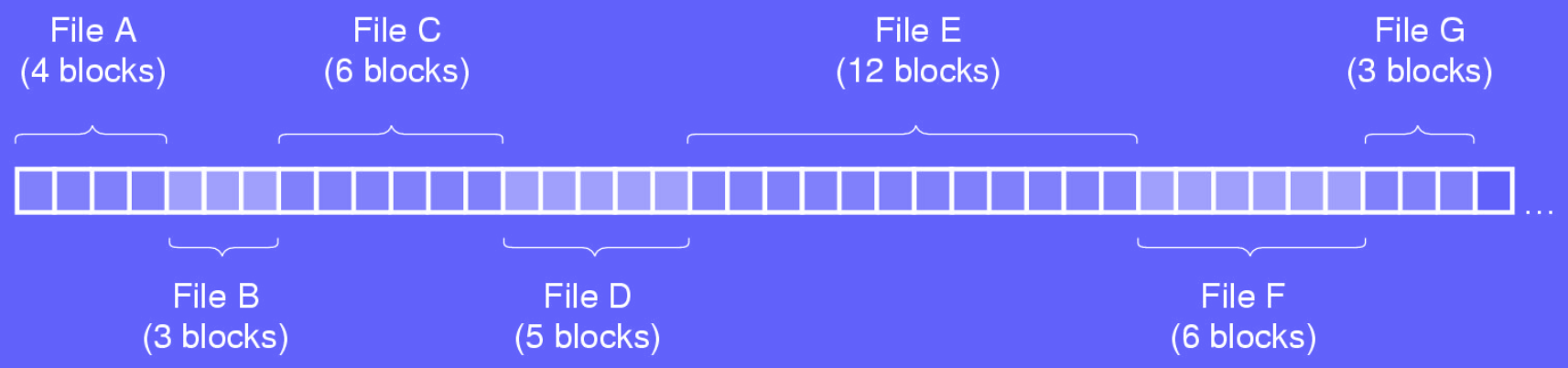
Contiguous allocation



Advantages:

- Simple to implement
- Fast: read a file using a single disk operation

Contiguous allocation



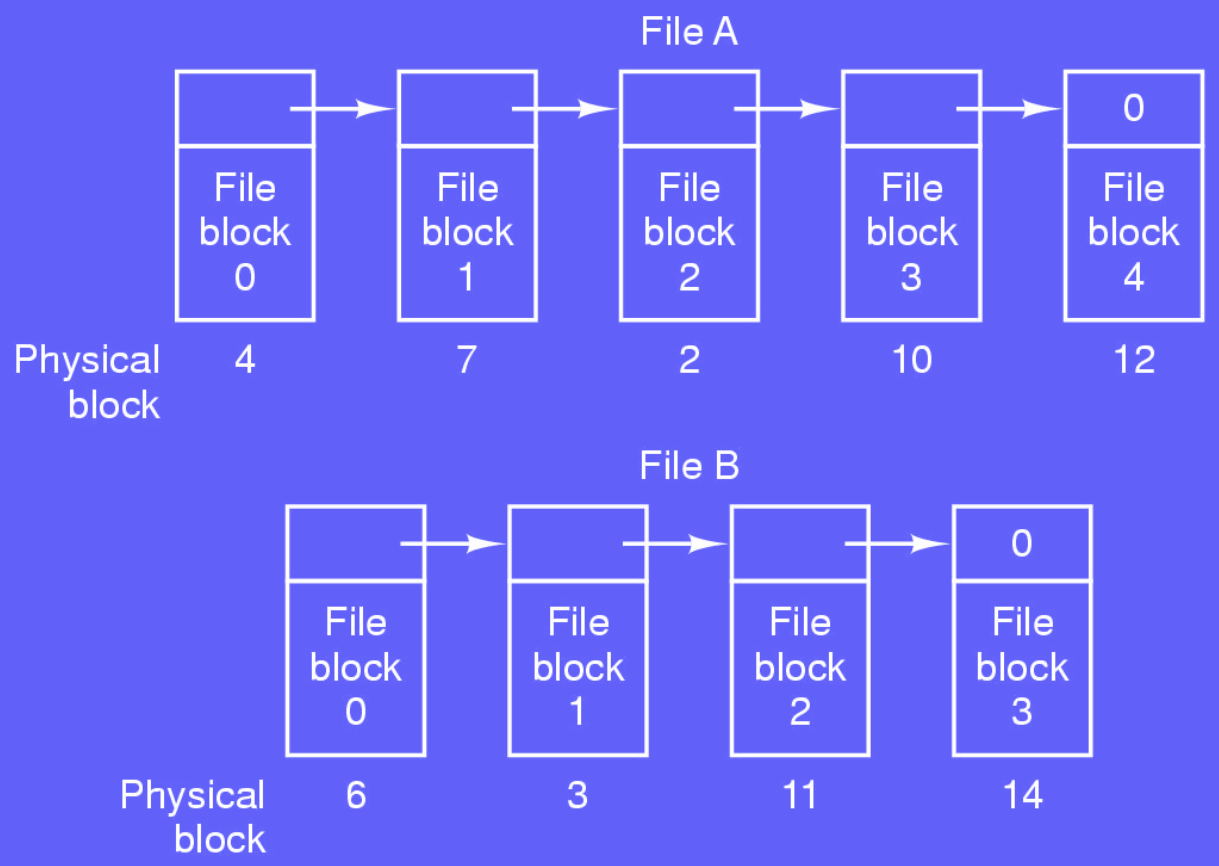
Advantages:

- Simple to implement
- Fast: read a file using a single disk operation

Drawback: what if files D and F are deleted?

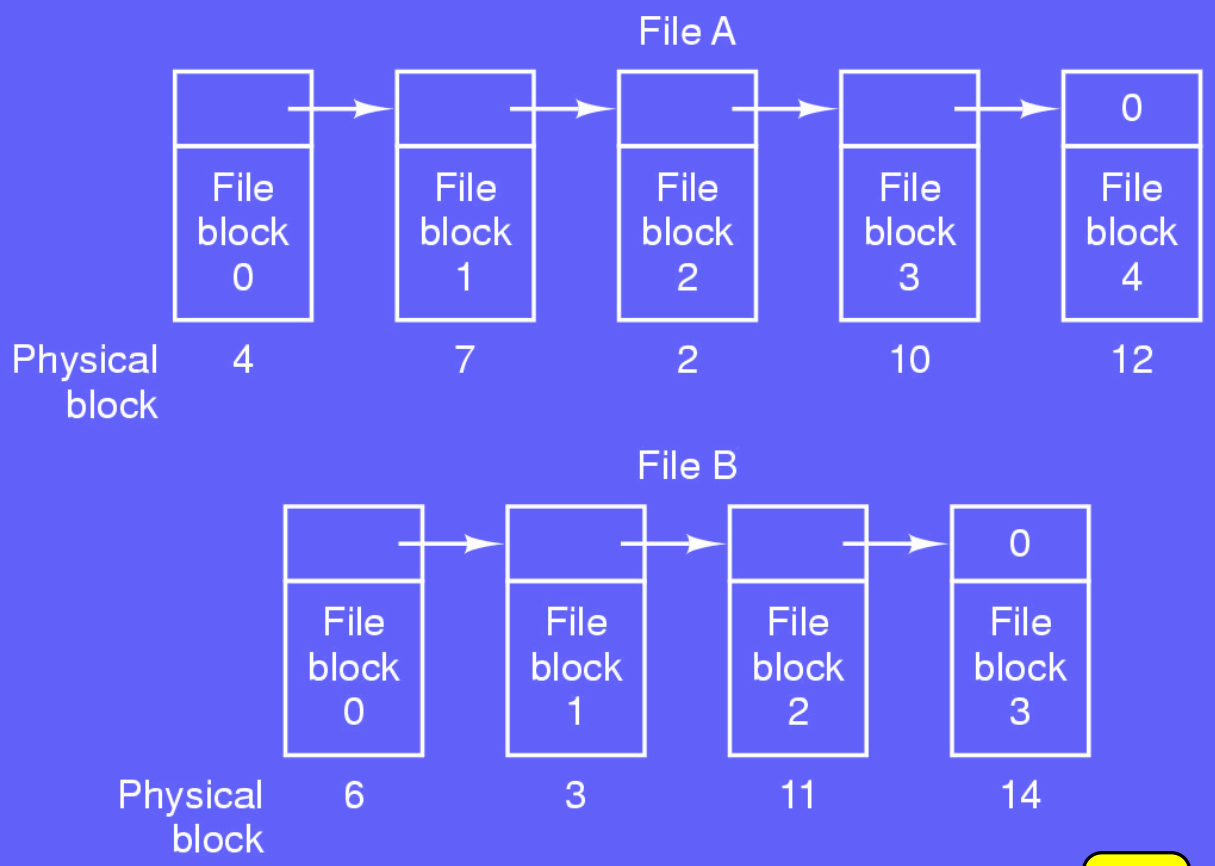
have holes

Linked list



Advantage: no fragmentation

Linked list



Advantage: no fragmentation

Drawback: slow random access have to go through all the blocks

File Allocation Table

Physical block		
0		
1		
2	10	
3	11	
4	7	← File A starts here
5		
6	3	← File B starts here
7	2	
8		
9		
10	12	
11	14	
12	-1	
13		
14	-1	
15		← Unused block

file B: 6 -> 3 -> 11 -> 14 (4 blocks)

Idea: save the pointers on all the disk blocks inside a table in the main memory

Advantage: fast random access

File Allocation Table

windows FAT

Physical block		
0		
1		
2	10	
3	11	
4	7	← File A starts here
5		
6	3	← File B starts here
7	2	
8		
9		
10	12	
11	14	
12	-1	
13		
14	-1	
15		← Unused block

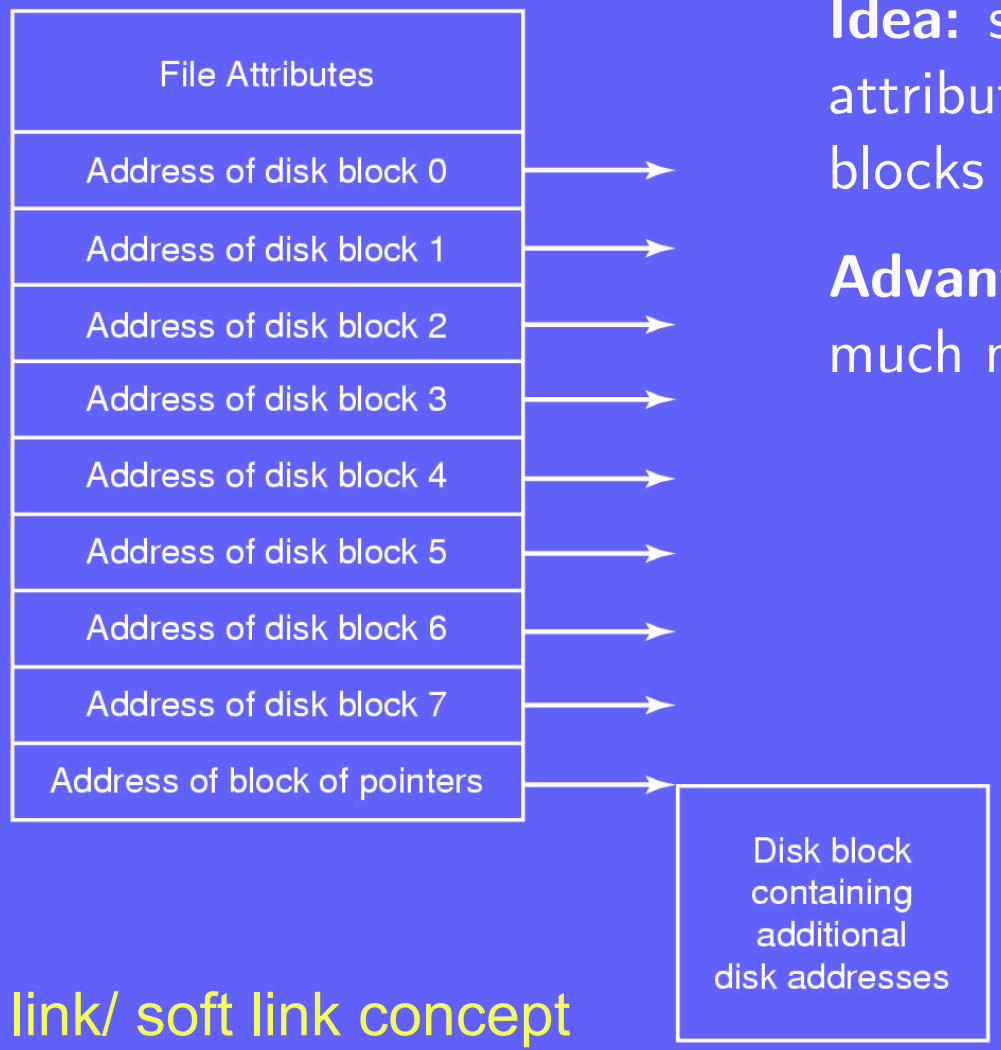
Idea: save the pointers on all the disk blocks inside a table in the main memory

Advantage: fast random access

Drawback: memory usage

very large memory to store this table

Index node



Idea: structure containing the file attributes and pointers on the blocks where the file is written

Advantage: fast, do not require much memory

hard link/ soft link concept

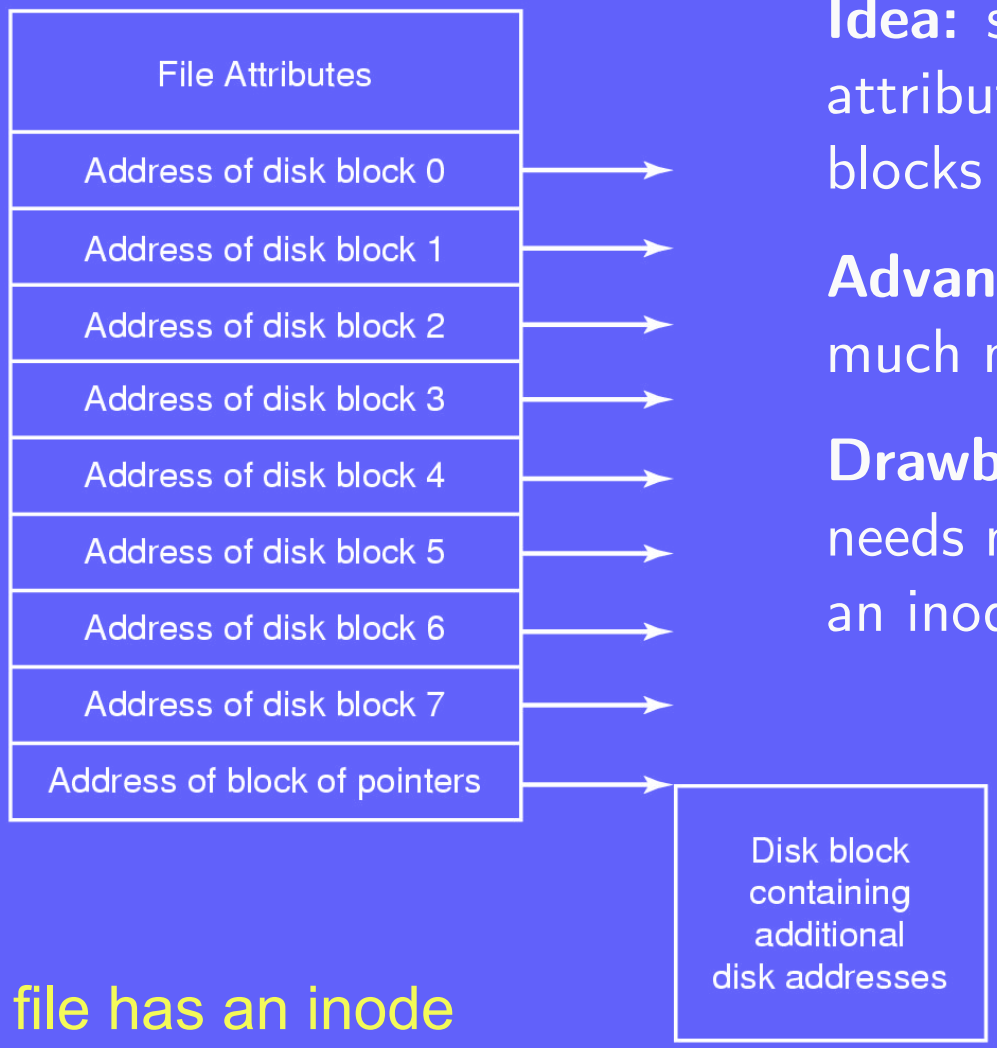
chattr: change attribute

Index node

**What is inode?

file attributes, address of blocks, idrect table..

.



Idea: structure containing the file attributes and pointers on the blocks where the file is written

Advantage: fast, do not require much memory

Drawback: what if a large file needs more blocks that can fit in an inode? add more blocks

each file has an inode

fs: free inodes/ occupied inodes

Directories

Simple design: fixed size entry (filename, attributes, disk address)

Directories

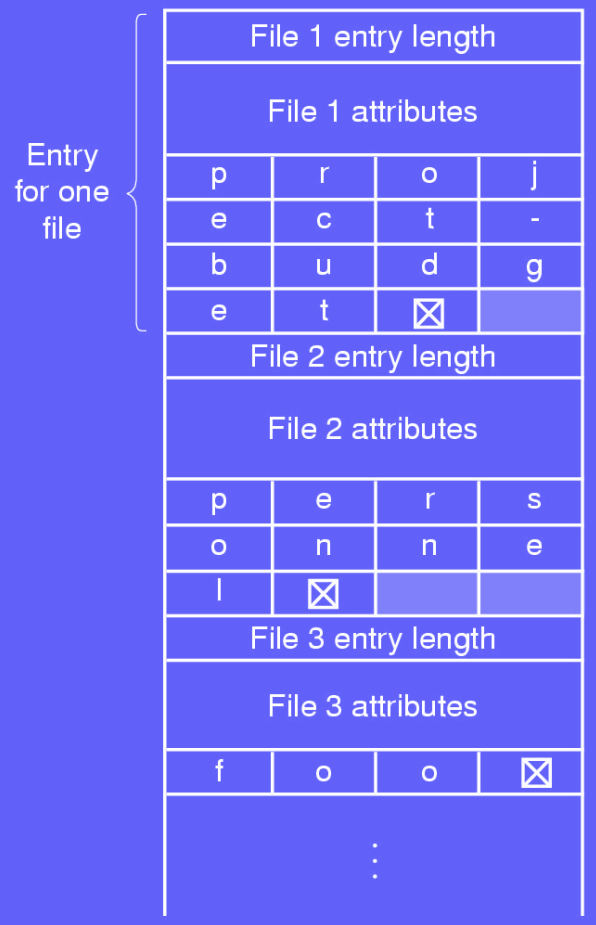
Simple design: fixed size entry (filename, attributes, disk address)

Drawback: how to handled long filenames?

Directories

Simple design: fixed size entry (filename, attributes, disk address)

Drawback: how to handled long filenames?



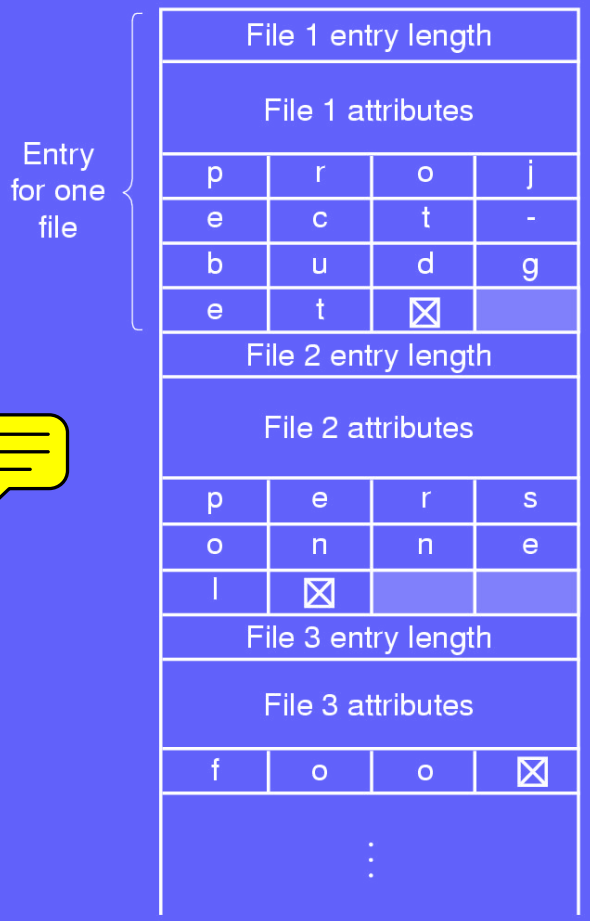
Idea: filename length not fixed

Advantage: can fit filename of arbitrary length

Directories

Simple design: fixed size entry (filename, attributes, disk address)

Drawback: how to handled long filenames?

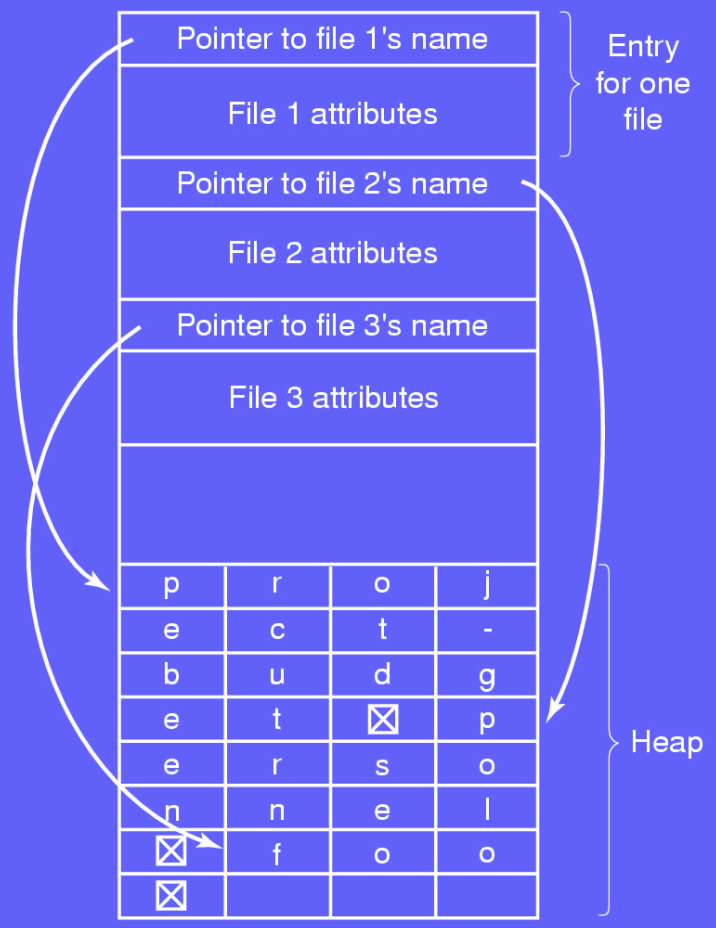


Idea: filename length not fixed

Advantage: can fit filename of arbitrary length

Drawback: space wasted, what if a directory entry spans multiple pages?

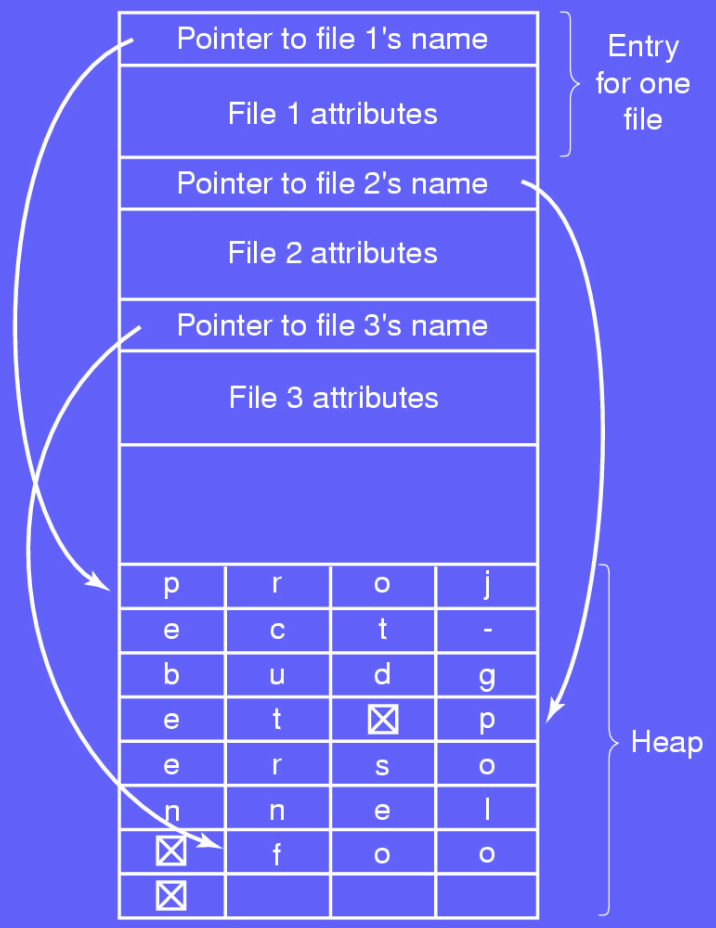
Directories



Idea: pointer to the filename

Advantage: no waste of space, space can be easily reused when a file is removed

Directories



Idea: pointer to the filename

Advantage: no waste of space, space can be easily reused when a file is removed

Drawback: as all the other strategies: slow on long directories

Journaling FS

Basic idea: log the operation to be performed, run it, and erase the log

Strategy: if a crash interrupts an operation, re-run it on next boot

Journaling FS

Basic idea: log the operation to be performed, run it, and erase the log

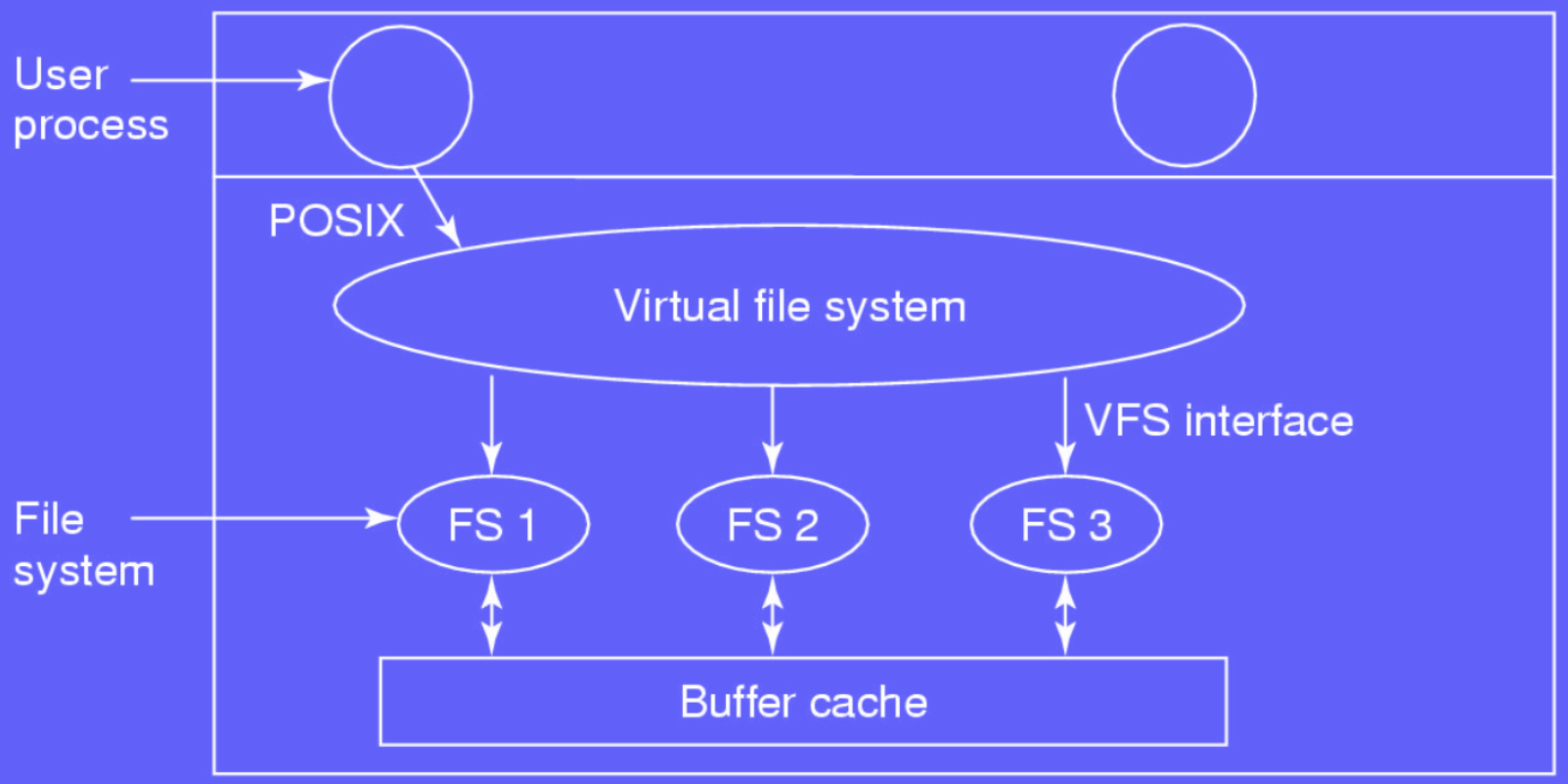
Strategy: if a crash interrupts an operation, re-run it on next boot

Problem: can any operation be applied more than once?

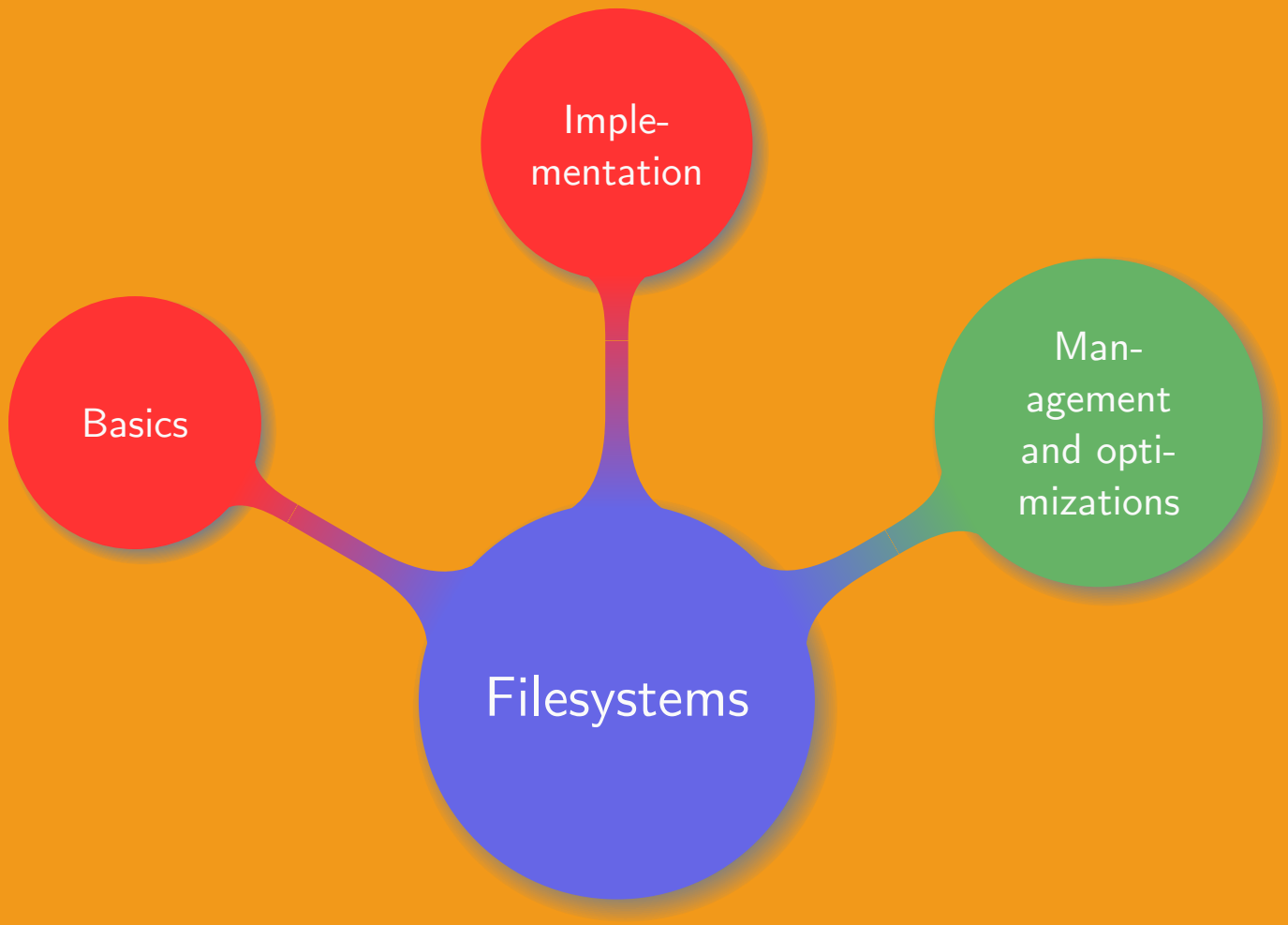
Example.
File deletion:

- (i) remove file from directory, (ii) release its i-node and (iii) add its disk blocks to the list of free blocks
- Operations (i) and (ii) can be repeated not (iii)

Virtual FS


User
process
POSIX
Virtual file system
VFS interface
File
system
FS 1
FS 2
FS 3
Buffer cache

Chapter organisation



Block size

Problem: how big should a block be?

Using small blocks:

- Large files use many blocks
- Blocks are not contiguous

Conclusion: time wasted

Block size

Problem: how big should a block be?

Using small blocks:

- Large files use many blocks
- Blocks are not contiguous

Conclusion: time wasted

Using large blocks:

- Small files do not fill up the blocks
- Many blocks partially empty

Conclusion: space wasted

Free blocks

Problem: how to keep track of free blocks?

Free blocks

4KB/64bits = 512 (pointers)
但是实际上只能存511个，因为一个用来从指向下一个list的指针

Problem: how to keep track of free blocks?

- *Using a linked list*: free blocks addresses are stored in a block e.g. using 4KB blocks with 64 bits block address, how many free blocks addresses can be stored in a block?
- *Using a bitmap*: one bit corresponds to one free block
- *Using consecutive free blocks*: a starting block and the number of free block following it

Which strategy is best?

FS consistency

fsck //file system check

Checking the FS:

- Using the i-nodes, list in all the blocks used by all the files. Compare the complementary to the list of free blocks
- For every i-node in every directory increment a counter by 1. Compare those numbers with the counts stored in the i-nodes

more inodes than files? not allowed (like mem leak)
hard link -> share the same inode -> more files than inodes

FS consistency

Checking the FS:

- Using the i-nodes, list in all the blocks used by all the files. Compare the complementary to the list of free blocks
- For every i-node in every directory increment a counter by 1. Compare those numbers with the counts stored in the i-nodes

Common problems and solutions:

- Block related inconsistency:
  - List of free blocks is missing some blocks → add blocks to list
  - Free blocks appear more than once in list → remove duplicates
  - A block is present in more than one file → copy block and add it to the files
- File related inconsistency:
  - Count in i-node is higher → set link count to accurate value
  - Count in i-node is lower → set link count to accurate value

Caching

Idea: keep in memory some disk blocks using the LRU algorithm

Questions:

- Is a block likely to be reused soon?

If not going to reuse soon, waste of mem

- What happens on a crash?

data lost, if cache data is not written into the main disk blocks

Modified idea:

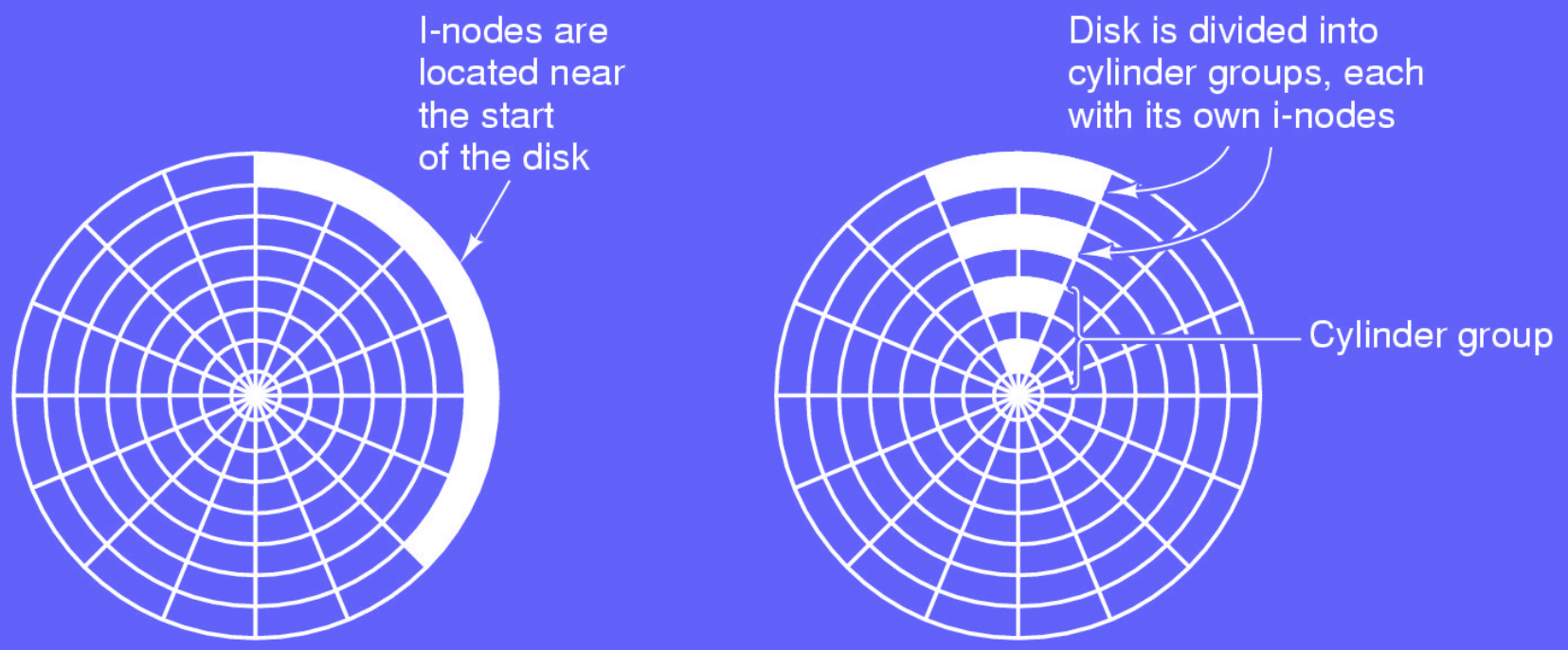
- Useless to cache i-node blocks

in case of crash. want to synchronize as soon as possible

- Dangerous to cache blocks essential to file system consistency
- Cache partially full blocks that are being written

Arm motion

Another way to increase file-system performance is by reducing the disk-arm motion by putting blocks that are likely to be accessed in sequence close to each other,preferably in the same cylinder.



In the left figure all the i-nodes are near the start of the disk, so the average distance between an inode and its blocks will be half the number of cylinders, requiring long seeks. But to increase the performance the placement of i-nodes can be modified as right.

Notes

A few extra remarks related to file systems:

- Quotas: assign disk quotas to users
- Fragmentation: how useful is it to defragment a file system?
- Block read ahead: when reading block k assume $k + 1$ will soon be needed and ensure its presence in the cache
- Logical volumes: file system over several disks
- Backups: how to efficiently backup a whole filesystem?
- RAID: Redundant Arrays of Inexpensive Disks

Key points

- What are the three main goals of a file system?
- Describe a basic disk layout
- Explain the structure of an i-node
- Mention three challenges in the design of a file system

Thank you!